RQ1 TECHNICAL REFERENCE

Where the Grounding Formulae Come From

A formula-to-PDDL guide for Routly's five model-side configurations

Reference case: 500 requested nodes, hybrid congestion, after macro-road preprocessing.
Purpose: explain every term shown on presentation slide 16, connect it to the generated PDDL, and clarify what the formula can and cannot say about runtime.

Contents

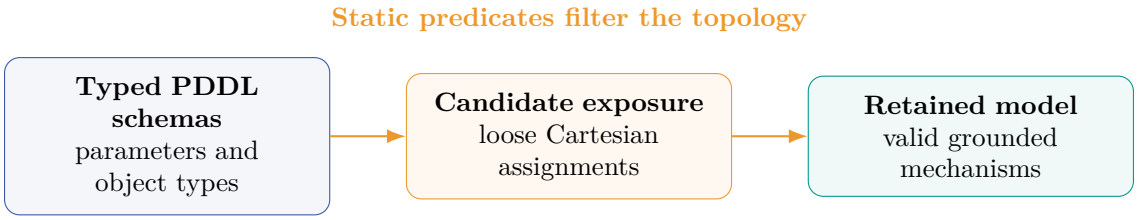
1 How to read slide 16

Key point

Slide 16 is **not a direct runtime predictor**. It separates two quantities that contribute to grounding in different ways: **candidate exposure** and **retained grounded structures**.

Candidates is a deliberately loose, type-based Cartesian upper bound. It counts assignments that are compatible with the declared PDDL types before static predicates such as `connects`, `road-next`, `static-road`, `goal-road`, and `next-window` remove impossible combinations. **Retained (derived)** is the number of valid grounded mechanisms left after static filtering. These are not the actions executed in the final plan. They are the actions, events, and processes made available to the subsequent search.

1.1 The two-stage interpretation



Candidate exposure describes the potential cost of *reaching* the valid model. Retained structures describe the cost of materialising, storing, and preprocessing the valid model that remains. The two quantities are related, but they are not added linearly and neither one alone predicts elapsed time.

1.2 Reference-case notation

Table 1: Symbols used throughout the derivations.

Symbol	Value	Meaning
R	829	Road objects after macro-road preprocessing.
L	483	Location objects after macro-road preprocessing.
W	21	Global congestion windows.
S	782	Roads selected as static by hybrid congestion.
D	47	Roads retained as dynamic; $S + D = R$.
T_s	1,471	Valid line-graph transitions whose current road is static.
T_d	79	Valid line-graph transitions whose current road is dynamic.
A	—	Retained action.
E	—	Retained event.
P	—	Retained process.

The experiment contains one vehicle. A vehicle factor would therefore be $V = 1$, so it is omitted from all slide formulae.

1.3 Three reusable counting rules

1. A schema with parameters `?r - road ?from - location ?to - location` exposes RL^2 typed assignments before `connects` is evaluated.

2. A static schema contributes one copy of the topology product. A dynamic schema with parameter `?w - time-window` contributes W copies. Together they produce the multiplier $1 + W$.
3. The event below has two window parameters. Its loose exposure is W^2, but the static chain `next-window` retains only adjacent pairs, hence $W - 1 = 20$ events.

```
(:event advance-window
 :parameters (?from - time-window ?to - time-window)
 :precondition (and
   (current-window ?from)
   (next-window ?from ?to)
   (>= (sim-time) (window-start ?to)))
 :effect (and
   (not (current-window ?from))
   (current-window ?to)))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_events`

$$\underbrace{W^2}_{21^2=441\text{ candidates}} \xrightarrow{\texttt{next-window}} \underbrace{W-1}_{20\text{ retained events}} .$$

2 PNP: process, node-based, parameterized

PNP **Process + node-based + parameterized**

The expressive baseline: ENHSP receives generic road/location schemas and represents the vehicle while it is travelling along a road.

COMPACT SLIDE FORMULA

$$\boldsymbol{RL^2(W+3)+W^2+1}$$

$= 4{,}641{,}518{,}386 \approx 4.642 \times 10^9$ candidates

2.1 Expanded derivation

$$\underbrace{RL^2(1+W)}_{\text{start actions}} + \underbrace{2RL^2}_{\text{arrival events}} + \underbrace{W^2}_{\text{advance-window event}} + \underbrace{1}_{\text{continuous process}}$$

The compact term $W + 3$ is simply:

$$W + 3 = (1 + W) + 2.$$

Table 2: PNP candidate terms and their PDDL origin.

Term	Origin	Reference value
RL^2	Static `start-traversal-static`: road, origin, destination.	193,396,581
RL^2W	Dynamic `start-traversal-dynamic`: the same parameters plus a window.	4,061,328,201
$2RL^2$	Two generic arrival schemas: normal intersection and traffic-light intersection.	386,793,162
W^2	`advance-window` binds a from-window and a to-window.	441
1	`traverse` has only one vehicle parameter and $V = 1$.	1

2.2 PDDL correspondence

```
(:action start-traversal-static
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location)
 :precondition (and
   (at ?v ?from) (connects ?r ?from ?to)
   (road-open ?r) (static-road ?r) (not (moving ?v)))
 :effect (and
   (not (at ?v ?from)) (on-road ?v ?r) (moving ?v)
   (assign (distance-remaining ?v) (road-length ?r))))

(:action start-traversal-dynamic
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location
              ?w - time-window)
 :precondition (and
   (at ?v ?from) (connects ?r ?from ?to)
   (road-open ?r) (dynamic-road ?r) (current-window ?w)
   (not (moving ?v)))
 :effect (and
   (not (at ?v ?from)) (on-road ?v ?r) (moving ?v)
   (assign (distance-remaining ?v) (road-length ?r))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_process_start_action`

```
(:process traverse
 :parameters (?v - vehicle)
 :precondition (and
   (moving ?v) (> (distance-remaining ?v) 0))
 :effect (and
   (decrease (distance-remaining ?v) (* #t (speed ?v)))
   (increase (total-distance ?v) (* #t (speed ?v)))
   (increase (sim-time) (* #t 1))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_process`

```
(:event arrive
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location)
 :precondition (and
   (on-road ?v ?r) (moving ?v)
   (connects ?r ?from ?to)
   (not (has-traffic-light ?to))
   (<= (distance-remaining ?v) 0))
 :effect (and
   (not (on-road ?v ?r)) (not (moving ?v)) (at ?v ?to)))

(:event arrive-at-traffic-light
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location)
 :precondition (and
   (on-road ?v ?r) (moving ?v)
   (connects ?r ?from ?to)
   (has-traffic-light ?to)
   (<= (distance-remaining ?v) 0))
 :effect (and
   (not (on-road ?v ?r)) (not (moving ?v)) (at ?v ?to)
   (increase (travel-time ?v) (light-wait ?to))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_events`

2.3 From candidates to retained structures

Static predicates select the real topology:

$$\underbrace{S + DW}_{782+47\cdot 21=1{,}769\ A} + \underbrace{R}_{829\ \text{arrival events}} + \underbrace{(W-1)}_{20\ \text{window events}} = 849E.$$

There is one vehicle process:

$$\boxed{1{,}769A + 849E + 1P = 2{,}619 \text{ retained mechanisms.}}$$

Although two arrival schemas exist, each real road ends either at a signalised node or at a non-signalised node. Static filtering therefore retains one arrival event per road, not two.

Interpretation

PNP is large for two independent reasons: the generic node-based schemas expose the RL^2 Cartesian product, and the process semantics retain arrival events plus a continuous process in the final grounded model.

3 CNP: compiled duration, node-based, parameterized

CNP **Compiled duration + node-based + parameterized**

The traversal semantics are compiled into one numeric action, but ENHSP still receives generic road-/from/to parameters.

COMPACT SLIDE FORMULA

$$\boldsymbol{RL^2(1+W)+W^2}$$

$$= 4{,}254{,}725{,}223 \approx 4.255 \times 10^9 \text{ candidates}$$

3.1 What disappears relative to PNP

$$\text{PNP} = \underbrace{RL^2(1+W)}_{\text{start actions}} + \underbrace{2RL^2}_{\text{arrival events}} + \underbrace{W^2}_{\text{window event}} + \underbrace{1}_{\text{process}},$$

$$\text{CNP} = \underbrace{RL^2(1+W)}_{\text{compiled traversal actions}} + \underbrace{W^2}_{\text{window event}}.$$

Compiled duration removes the two generic arrival-event schemas and the continuous process. It does *not* remove the dominant road/from/to Cartesian product because action generation remains parameterized.

3.2 PDDL correspondence

```
(:action traverse-road-static
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location)
 :precondition (and
   (at ?v ?from) (connects ?r ?from ?to)
   (road-open ?r) (static-road ?r))
 :effect (and
   (not (at ?v ?from)) (at ?v ?to)
   (increase (travel-time ?v) (travel-duration ?r))
   (increase (sim-time) (travel-duration ?r))))

(:action traverse-road-dynamic
 :parameters (?v - vehicle ?r - road
              ?from - location ?to - location
              ?w - time-window)
 :precondition (and
   (at ?v ?from) (connects ?r ?from ?to)
   (road-open ?r) (dynamic-road ?r) (current-window ?w))
 :effect (and
   (not (at ?v ?from)) (at ?v ?to)
   (increase (travel-time ?v) (travel-duration-window ?r ?w))
```

```
  (increase (sim-time) (travel-duration-window ?r ?w))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_compiled_duration_action`

There is no hidden process maintaining time. The action directly increases `sim-time`. Once it crosses the start of the next window, the generic `advance-window` event becomes applicable.

3.3 Numeric derivation

$$\begin{aligned} RL^2(1+W) &= 829 \cdot 483^2 \cdot 22 \\ &= 4{,}254{,}724{,}782, \\ W^2 &= 21^2 = 441, \\ \text{total} &= 4{,}254{,}725{,}223. \end{aligned}$$

After static filtering:

$$\boxed{1{,}769A + 20E = 1{,}789 \text{ retained mechanisms.}}$$

Interpretation

CNP is structurally lighter than PNP because compiled duration removes the arrival events and the process. However, its candidate exposure remains in the same order of magnitude because `?r`, `?from`, and `?to` are still generic.

4 CNC: compiled duration, node-based, compiled

CNC — **Compiled duration + node-based + compiled**

Python instantiates the valid road/end-point combinations and uses singleton types so that each specialised parameter has exactly one compatible object.

COMPACT SLIDE FORMULA

$$S + DW + W^2$$

$= 2{,}210$ candidates

4.1 From a generic action to one specialised schema

Same retained model, different candidate exposure

CNP: one generic schema
ENHSP binds road, from, to, and possibly window

→

CNC: many specialised schemas
Python selects one valid road/end-point/window tuple

```
(:types
  vehicle location road time-window - object
  loc_type_loc_a loc_type_loc_b - location
  road_type_road_0001 - road
  window_type_tw_00000 - time-window)

(:objects
  vehicle_0 - vehicle
  loc_a - loc_type_loc_a
  loc_b - loc_type_loc_b
  road_0001 - road_type_road_0001
  tw_00000 - window_type_tw_00000)
```

Generator origin: `domain_generator.py :: _build_types;`
`problem_generator.py :: _build_objects`

```
(:action traverse-road-dynamic-road_0001-tw_00000
 :parameters (?v - vehicle
              ?r - road_type_road_0001
              ?from - loc_type_loc_a
              ?to - loc_type_loc_b
              ?w - window_type_tw_00000)
 :precondition (and
   (at ?v ?from) (connects ?r ?from ?to)
   (road-open ?r) (dynamic-road ?r) (current-window ?w))
 :effect (and
   (not (at ?v ?from)) (at ?v ?to)
   (increase (travel-time ?v) (travel-duration-window ?r ?w))
   (increase (sim-time) (travel-duration-window ?r ?w))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_compiled_duration_road_actions`

The action still has normal PDDL parameters. The difference is that every specialised type contains one object, so this schema exposes one legal binding instead of a generic Cartesian product.

4.2 Candidate derivation

$$\underbrace{S}_{\substack{\text{one specialised schema}\\ \text{per static road}}} + \underbrace{DW}_{\substack{\text{one specialised schema}\\ \text{per dynamic road/window}}} + \underbrace{W^2}_{\text{generic advance-window}}$$

$$782 + (47 \cdot 21) + 21^2 = 782 + 987 + 441 = \boxed{2{,}210}.$$

The first two terms are already valid by construction. Only `advance-window` remains generic:

$$\boxed{1{,}769A + 20E = 1{,}789 \text{ retained mechanisms.}}$$

Key point

CNP and CNC retain the same final model: $1{,}769A + 20E$. The difference is the path used to reach it. CNP exposes roughly 4.255 billion typed assignments; CNC exposes 2,210.

Interpretation

This is the cleanest evidence for singleton types. They do not change which movements are available after grounding; they move the valid instantiation from ENHSP to Python and collapse the generic binding space.

5 CLP: compiled duration, line graph, parameterized

CLP — **Compiled duration + line graph + parameterized**

The vehicle state is attached to the current road. A generic action chooses the current road and a possible successor road.

COMPACT SLIDE FORMULA

$$\boldsymbol{R(R+1)(1+W)+W^2}$$

$$= 15{,}137{,}981 \approx 1.514 \times 10^7 \text{ candidates}$$

5.1 Expanded derivation

$$\underbrace{R^2(1+W)}_{\text{road-to-road traversal}} + \underbrace{R(1+W)}_{\text{finish-road}} + \underbrace{W^2}_{\text{advance-window}}$$

$$R^2(1+W) + R(1+W) = R(R+1)(1+W).$$

Table 3: CLP candidate terms and their PDDL origin.

Term	Origin
R^2	`traverse-road-static` binds a current road and a possible next road.
R^2W	`traverse-road-dynamic` adds a window parameter.
R	`finish-road-static` binds only the current goal road candidate.
RW	`finish-road-dynamic` adds a window parameter.
W^2	The shared generic `advance-window` event.

5.2 PDDL correspondence

```
(:action traverse-road-static
 :parameters (?v - vehicle ?r - road ?next - road)
 :precondition (and
   (ready-road ?v ?r) (road-next ?r ?next)
   (road-open ?r) (static-road ?r))
 :effect (and
   (not (ready-road ?v ?r)) (ready-road ?v ?next)
   (increase (travel-time ?v) (travel-duration ?r))
   (increase (sim-time) (travel-duration ?r))))

(:action traverse-road-dynamic
 :parameters (?v - vehicle ?r - road ?next - road
              ?w - time-window)
 :precondition (and
   (ready-road ?v ?r) (road-next ?r ?next)
   (road-open ?r) (dynamic-road ?r) (current-window ?w))
 :effect (and
   (not (ready-road ?v ?r)) (ready-road ?v ?next)
   (increase (travel-time ?v) (travel-duration-window ?r ?w))
   (increase (sim-time) (travel-duration-window ?r ?w))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_compiled_duration_line_graph_action`

```
(:action finish-road-static
 :parameters (?v - vehicle ?r - road)
 :precondition (and
   (ready-road ?v ?r) (goal-road ?r)
   (road-open ?r) (static-road ?r))
 :effect (and
   (not (ready-road ?v ?r)) (reached-goal ?v)
   (increase (travel-time ?v) (travel-duration ?r))
   (increase (sim-time) (travel-duration ?r))))

(:action finish-road-dynamic
 :parameters (?v - vehicle ?r - road ?w - time-window)
 :precondition (and
   (ready-road ?v ?r) (goal-road ?r)
   (road-open ?r) (dynamic-road ?r) (current-window ?w))
 :effect (and
   (not (ready-road ?v ?r)) (reached-goal ?v)
   (increase (travel-time ?v) (travel-duration-window ?r ?w))
   (increase (sim-time) (travel-duration-window ?r ?w))))
```

Generator origin: `src/routly/pddl/domain_generator.py :: _build_compiled_duration_line_graph_action`

5.3 Numeric derivation and static filtering

$$\begin{aligned} R(R+1)(1+W) &= 829 \cdot 830 \cdot 22 \\ &= 15{,}137{,}540, \\ W^2 &= 441, \\ \text{total} &= 15{,}137{,}981. \end{aligned}$$

The problem generator creates `road-next` only for true graph successors:

```
(road-next road_0001 road_0007)
(road-next road_0001 road_0012)
...
(goal-road road_goal)
```

Generator origin: `src/routly/pddl/problem_generator.py :: _build_init`

After static filtering:

$$\underbrace{T_s + T_d W}_{\substack{1{,}471+79\cdot 21=3{,}130 \\ \text{traversals}}} + \underbrace{1}_{\substack{\text{valid} \\ \text{finish-road}}} + \underbrace{20}_{\substack{\text{window} \\ \text{events}}} = \boxed{3{,}151}.$$

The selected goal road is static in this reference instance, so one finish action is retained. If the selected goal road were dynamic, its retained finish contribution would be W rather than 1.

Interpretation

The line graph reduces parameter arity and therefore candidate exposure. However, it retains more actions than the node-based encoding because one current road may have several valid successor roads.

6 CLC: compiled duration, line graph, compiled

CLC **Compiled duration + line graph + compiled**

Python enumerates only valid road-to-road transitions and assigns singleton types to the current road, successor road, and, when required, window.

COMPACT SLIDE FORMULA

$$\boldsymbol{T_s + T_d W + 1 + W^2}$$

$= 3{,}572$ candidates

6.1 Specialised line-graph action

```
(:action traverse-road-dynamic-road_0001-road_0007-tw_00000
 :parameters (?v - vehicle
              ?r - road_type_road_0001
              ?next - road_type_road_0007
              ?w - window_type_tw_00000)
 :precondition (and
   (ready-road ?v ?r) (road-next ?r ?next)
   (road-open ?r) (dynamic-road ?r) (current-window ?w))
 :effect (and
   (not (ready-road ?v ?r)) (ready-road ?v ?next)
   (increase (travel-time ?v) (travel-duration-window ?r ?w))
   (increase (sim-time) (travel-duration-window ?r ?w))))
```

Generator origin: `domain_generator.py :: _build_compiled_duration_line_graph_road_actions`

```
(:action finish-road-static-road_goal
 :parameters (?v - vehicle ?r - road_type_road_goal)
 :precondition (and
   (ready-road ?v ?r) (goal-road ?r)
   (road-open ?r) (static-road ?r))
 :effect (and
   (not (ready-road ?v ?r)) (reached-goal ?v)
   (increase (travel-time ?v) (travel-duration ?r))
   (increase (sim-time) (travel-duration ?r))))
```

Generator origin: `domain_generator.py :: _build_compiled_duration_line_graph_road_actions`

6.2 Candidate derivation

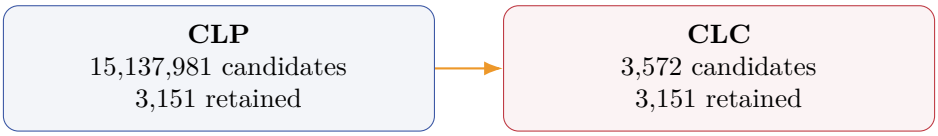
$$\underbrace{T_s}_{\text{static valid transitions}} + \underbrace{T_d W}_{\text{dynamic valid transition/window pairs}} + \underbrace{1}_{\text{static goal-road finish}} + \underbrace{W^2}_{\text{generic window event}}.$$

$$1{,}471 + (79 \cdot 21) + 1 + 441 = 1{,}471 + 1{,}659 + 1 + 441 = \boxed{3{,}572}.$$

After `next-window` filtering:

$$\boxed{3{,}130 \text{ traversal actions} + 1 \text{ finish action} + 20 \text{ events} = 3{,}151.}$$

Same retained model, different candidate exposure



Interpretation

CLP and CLC retain the same grounded model. CLC is faster because singleton types remove almost all generic binding exploration, not because it removes valid line-graph transitions.

7 All five configurations side by side

Table 4: Complete 500-node hybrid reference-case decomposition.

Code	Candidate formula	Candidates	Retained
PNP	$RL^2(1+W) + 2RL^2 + W^2 + 1$	4.642×10^9	2,619
CNP	$RL^2(1+W) + W^2$	4.255×10^9	1,789
CNC	$S + DW + W^2$	2,210	1,789
CLP	$R^2(1+W) + R(1+W) + W^2$	1.514×10^7	3,151
CLC	$T_s + T_d W + 1 + W^2$	3,572	3,151

7.1 What each modelling axis changes

7.2 The two strongest controlled comparisons

CNP versus CNC: same node-based state and the same retained model (1,789), but candidates fall from about 4.255 billion to 2,210. This isolates action generation.
CLP versus CLC: same line-graph state and the same retained model (3,151), but candidates

Axis	Structural effect	Visible comparison
Traversal	Compiled duration removes process and arrival-event semantics.	PNP → CNP
State	Line graph replaces two location parameters with a current/next-road pair, but may retain more transitions because of branching.	CNP ↔ CLP
Generation	Singleton types move valid instantiation from ENHSP to Python.	CNP → CNC; CLP → CLC

fall from about 15.14 million to 3,572. This independently confirms the singleton-type effect.

Do not compare the slide-16 retained counts directly with the slide-17 runtimes. Slide 16 uses a 500-node hybrid instance and derived counts because PNP, CNP, and CLP do not finish at that scale. Slide 17 uses a 300-node static instance on which all five configurations terminate.

8 Why the formulae do not directly predict grounding time

8.1 The 300-node static experiment

On the fixed static instance, the retained grounded structures are:

$$\text{PNP} = 925 = 462A + 462E + 1P,$$

$$\text{CNP} = \text{CNC} = 462A, \qquad \text{CLP} = \text{CLC} = 851A.$$

Table 5: Measured grounding results on the fixed 300-node static instance.

Configuration	Grounding (s)	Retained mechanisms	Main explanation
PNP	79.253	925	Actions + arrival events + process
CNP	40.379	462	Generic node-based binding
CNC	1.063	462	Singleton node-based binding
CLP	76.263	851	Generic road-pair binding + branching
CLC	1.914	851	Singleton road-pair binding

8.2 The apparent CLP contradiction

On this 300-node static case, the loose type-based exposures are:

$$\text{PNP} = 3RL^2 + 1 = 121{,}272{,}493,$$

$$\text{CNP} = RL^2 = 40{,}424{,}164, \qquad \text{CLP} = R(R+1) = 234{,}740.$$

Looking only at candidates would suggest that CLP must ground faster than CNP. The measured result is the opposite: CLP takes 76.263 seconds, whereas CNP takes 40.379 seconds. The retained model explains why: CLP leaves 851 valid actions, while CNP leaves 462.

Looking only at retained structures would create a different contradiction: CLP and CLC both retain 851 actions, yet CLC grounds in 1.914 seconds. Candidate exposure explains the difference: CLC reaches the same valid model without generic road-pair enumeration.

Key point

Candidates explain the cost of reaching and filtering the model. Retained structures explain the cost of materialising and preprocessing the model that remains. Runtime depends on both, as well as on the mechanism types, numeric fluents, and ENHSP's internal implementation.

8.3 Why PNP and CLP have similar times

PNP and CLP arrive at similar retained model sizes for different reasons:

- PNP retains fewer movement actions, but also retains arrival events, one process, and the continuous mid-road state.
- CLP removes processes and arrival events, but line-graph branching leaves 851 road-to-road actions.

This is why the measured grounding times are also similar:

$$79.253 \text{ s (PNP)} \qquad \text{versus} \qquad 76.263 \text{ s (CLP)}.$$

9 Traceability to the implementation

Table 6: Where each formula component is implemented.

Component	Generator function	Counting role
PNP start actions	`_build_process_start_action`	$RL^2(1+W)$
PNP process	`_build_process`	$1P$ for one vehicle
Arrival and window events	`_build_events`	$2RL^2+W^2$ in PNP; W^2 otherwise
CNP generic actions	`_build_compiled_duration_action`	$RL^2(1+W)$
CNC specialised actions	`_build_compiled_duration_road_actions`	$S+DW$
CLP generic line graph	`_build_compiled_duration_line_graph_action`	$R^2(1+W)+R(1+W)$
CLC specialised line graph	`_build_compiled_duration_line_graph_road_actions`	T_s+T_dW+1
Singleton type declarations	`_build_types` and `_build_objects`	One compatible object per specialised parameter
Static topology facts	`_build_init`	`connects`, `road-next`, `goal-road`, `next-window`

Primary implementation files
`src/routly/pddl/domain_generator.py`
`src/routly/pddl/problem_generator.py`
Presentation source
`docs/latex_source/final_presentation/main.tex`, grounding slide.
Report discussion
`docs/latex_source/final_report/report.tex`, model-side scalability and three modelling axes. The submitted report is not modified by this technical reference.

Final one-sentence interpretation

Candidate exposure measures the potential Cartesian burden, while retained structures measure the grounded model left after static filtering. Neither quantity alone predicts runtime.